darkmpc, Milestone 4a walkthrough

Catching a cheating server or trader

Tave La Rocque

4 November 2026

Milestone 4a, step 1: catching a cheating server

What changed

Every multiplication and AND gate is now checked before anything is opened. With `malicious: true`, the parties compute exactly as before, but every product is recorded as a triple of shares (x, y, z) that should satisfy $z = xy$. Before any opening of real data, which happens inside the sort's permutation step and at the final delivery of fills, the servers prove every recorded product correct. They then agree on a verdict: each tells both others whether all its checks passed, and a single "no" anywhere makes every honest server abort. The semi-honest path is unchanged and costs nothing extra.

What this does not yet cover: the shuffle inside the sort (Milestone 4b) and checks on traders' inputs (step 2). Until both are done, the honest description is: "multiplications, AND gates and openings are actively secure with abort; the shuffle and input validation are not yet."

The check, from the bottom up

Opening with two copies. In replicated sharing each server is missing exactly one summand, and the other two servers both hold it. A checked opening has both send it; the receiver compares the copies. With at most one corrupt server, one copy is honest, so a wrong value always shows up as a mismatch. Every opening inside verification is a checked opening.

Checking a product against a triple. Given a product (x, y, z) to check and a random triple (a, b, c) with $c = ab$, open $\rho = x - a$ and $\sigma = y - b$. These reveal nothing, because a and b are uniformly random and used once. Then compute locally

$$t = z - c - \sigma a - \rho b - \rho\sigma$$

and open t, which must be 0. The reason, line by line:

$$xy = (\rho + a)(\sigma + b)$$

because $x = \rho + a$ and $y = \sigma + b$ by definition of the masks;

$$= \rho\sigma + \rho b + \sigma a + ab$$

by expanding;

$$\Rightarrow \; z - c - \sigma a - \rho b - \rho\sigma = (z - xy) - (c - ab)$$

by substituting $ab = c - (c - ab)$. So if $z = xy + \delta_z$ and $c = ab + \delta_c$, the opened value is $t = \delta_z - \delta_c$. It is zero exactly when the two errors are equal. For AND gates the same algebra holds with XOR for both plus and minus, and AND for times. This is Protocol 2.24 of Furukawa, Lindell, Nof and Weinstein (Eurocrypt 2017).

Getting triples you can trust. Triples are made with the same semi-honest multiplication, so a cheater could corrupt them too. FLNW's answer (their Protocol 3.2) is cut-and-choose with buckets:

1. Make $NB + C$ triples.
2. Shuffle them with coins the servers toss together after the triples exist. The coins come from opening a random sharing with a checked opening, so no server can predict or bias them.
3. Open the first C triples completely and check $c = ab$ directly.
4. Deal the rest into N buckets of B, and check each bucket's first triple against the other $B - 1$.

A bad triple survives only if every triple in its bucket carries the same error and none of them was opened. Their Theorem 5.3 bounds the probability by $N/\binom{NB+C}{B}$ when $C \geq B$. A test reproduces their published example: $N = 2^{20}$ triples at 40-bit security needs $B = C = 3$.

Why this extends to $\mathbb{Z}_{2^{64}}$. FLNW's analysis covers Boolean circuits. Our AND words are exactly that: 64 independent gates per word. For ring multiplication, the argument is mine and goes as follows. A corrupt server's messages are functions of its own view. That view is independent of the secrets, because everything it receives is masked. So any deviation shifts a product by an additive error that does not depend on the secrets. A check passes only when two errors are equal. So the bucket game is the same as the Boolean one: a larger ring gives the cheater more error values to choose from, but no extra way to pass. The design document states this, and the final report will include the proof.

Bounded memory and an honest security budget

The first version checked everything at the next opening. At 10,000 orders it ran out of memory: all three elimination passes' comparisons, millions of AND words, piled up before the final unsort. Now products are verified in batches of at most 2^{18} as they accumulate. Checking earlier is never less secure: a failed batch only marks the server's local verdict false, and no data is opened before the next agreed verdict.

More batches means more chances for a cheater. So each batch targets 2^{-56} (FLNW's bound). An AND word is 64 gates checked together, which costs 6 bits by a union bound, and b batches cost another $\log_2 b$ bits. The run reports its own overall bound: 46 bits at 1,000 orders (16 batches), 44.8 at 10,000 (36 batches) and 42.0 at 100,000 (253 batches). That stays above 40 bits throughout, but the margin shrinks with batch count, so the bound is computed per run, not assumed.

Planted cheating servers

In each case party 1 cheats exactly once, and the honest servers must abort:

Plant	What the cheater does	What catches it
`plant-cheat-mul`	sends a wrong multiplication message	the product's check: $t \neq 0$
`plant-cheat-and`	flips an AND output bit	the AND gate's check
`plant-cheat-keep-own`	sends the right message but keeps a different value as its own copy	the two holders compute different shares of t: copy mismatch
`plant-cheat-triple`	corrupts one triple	either opened in the cut, or checked against a good triple in its bucket
`plant-cheat-open-copy`	sends a wrong copy during an opening	the receiver's copy comparison

All five are caught, and caught the right way. The failing assertion in each case is "honest run aborted": honest servers detected the cheat and stopped, rather than producing a wrong answer. With these, `check_planted.sh` now has 15 planted bugs, all caught.

Measured cost

All three servers on one 2-vCPU machine, 100 symbols, 3 passes. Every run matches the reference.

Orders	Mode	Rounds	Data per server	Time
1,000	semi-honest, in-process	316	6.9 MB	0.026 s
1,000	malicious, in-process	458	118.8 MB	0.833 s
1,000	malicious, TLS processes, simulated LAN	458	118.8 MB	1.124 s
10,000	malicious, TLS processes, simulated LAN	630	1.36 GB	11.05 s
100,000	semi-honest, in-process	372	0.80 GB	3.22 s
100,000	malicious, in-process	2,173	15.1 GB	92.8 s

Malicious mode costs about 17 to 19 times the semi-honest communication. Most of that is my implementation, not the method. Each verified product needs about B triple multiplications, plus $B-1$ bucket checks, plus its own check. Each check opens ρ, σ and t with both copies sent in full. FLNW avoid most of that: zero checks and second copies are compared through hashes of the views, not by sending every value twice. That is the first optimisation to make, and the benchmarks will be rerun after it.

The comparison with prior work, stated carefully

The 2018 paper's actively secure three-party volume match processed 1,000 orders in 0.900 s online plus 4.136 s of preprocessing, on three real machines on a LAN. Our malicious-mode run of 1,000 orders took 1.12 s end to end with no separate preprocessing, through TLS on a simulated LAN with all three servers sharing two CPUs. It also hides symbols and sides and handles minimum fills.

It is **not yet** a like-for-like claim. Our shuffle is not yet verified (4b), our inputs are not yet checked (step 2), and their machines were real while ours were simulated. Both gaps have to close before the comparison goes in the README.

Milestone 4a, step 2: catching a malicious trader

What a trader can do, and what stops it

A trader shares its own order, so it controls every value the servers start from. There are two ways to abuse that, and the servers now check both on shares before matching. Any order that fails is replaced, on every server, by a zero-size dummy with public values, so it takes part in nothing. Honest orders are unaffected: a dummy adds zero to every sum and every queue.

Inconsistent shares. In replicated sharing each summand goes to two servers. A trader could give those two servers different values, so that the three servers' pieces describe no single order and different servers compute on different secrets. The check costs one round: server i sends its copy of summand x_{i+1} to server $i+1$, which holds the same summand and compares. The servers then tell each other which orders failed. In a second round each server repeats to each neighbour what the other neighbour told it. A corrupt server that tells its two peers different things (equivocation) is caught there, and the auction aborts.

Honest limitation: a corrupt server can still *claim* a mismatch that did not happen, and so get an honest order rejected. It learns nothing by doing this, but it is censorship, and the threat model says so.

Values out of range. Three checks follow, and each opens only something that is a fixed public constant for an honest order.

- **Key bits** (the symbol bits and the side) must be 0 or 1. For a bit b, the value $b-b^2$ is 0 exactly when $b \in \{0,1\}$, so the servers compute it with one multiplication and open it. An honest order opens 0.

- **Sizes and minimum fills** must be below 2^{40}. Here the comparison circuit pays off a second time. The carry circuit already computes every carry, so the full bit decomposition costs nothing extra: $s + c = p \oplus (G \ll 1)$. The servers mask out bits 0 to 39 with a public AND and open what is left, bits 40 to 63. An honest order opens 0.
- **The order makes sense:** $1 \leq$ min fill $\leq$ size. Comparisons are sound once both values are below 2^{40}, so the servers compute $[0 < \text{size}]\,[0 < \text{min}]\,(1 - [\text{size} < \text{min}])$ and open it. An honest order opens 1.

The order of the checks matters. A size of 2^{63} passes the third check alone: $0 - 2^{63}$ wraps to a value whose top bit is set, so "$0 < $ size" comes out true. That is exactly why the range check runs first, and why it has its own planted bug.

Planted malicious traders, and planted gaps

Nine bad orders are tested, each placed among five honest ones: inconsistent copies, a size of 2^{63}, a size of 2^{40} (one past the limit), a size of all ones, a side of 2, a symbol bit of 2, a minimum fill of 0, a minimum fill above the size, and a size of 0. Two properties are checked for each. First, exactly the bad order is rejected, on all three servers. Second, the honest orders still get exactly the reference matcher's fills, with 0 in the bad order's slot.

Four planted bugs each switch off one check: consistency, bit check, range check and order check. Each one makes the tests fail. `check_planted.sh` now runs 19 planted bugs, and all 19 are caught.

Cost

Validation adds 62 rounds and about 25 MB per server at 1,000 orders in malicious mode: 520 rounds and 144 MB in total, 1.35 s through TLS processes on the simulated LAN, still matching the reference. In malicious mode it is always on. In semi-honest mode it is optional (`--validate`), because there the servers are trusted to follow the protocol but traders are not.